

AERO-SIM

An AI-enabled real-time digital twin for aero piston engines in MALE-class UAVs

Problem statement	DRDO PS 26054 / SIH 26054
Engines modelled	Rotax 912 ULS, 914 UL/F, 915 iS, 916 iS
Components	Physics twin · 5-head AI service · physics residuals · cockpit · CAN bridge
Repository	github.com/Harsh24856/AirSim (branches: main, ui)
Report date	17 September 2026

What this document is. A technical walkthrough of the system: what it simulates, how the AI reads it, how each claim was verified, and what is still open. Every number quoted here was measured on this build — none is an estimate or a target.

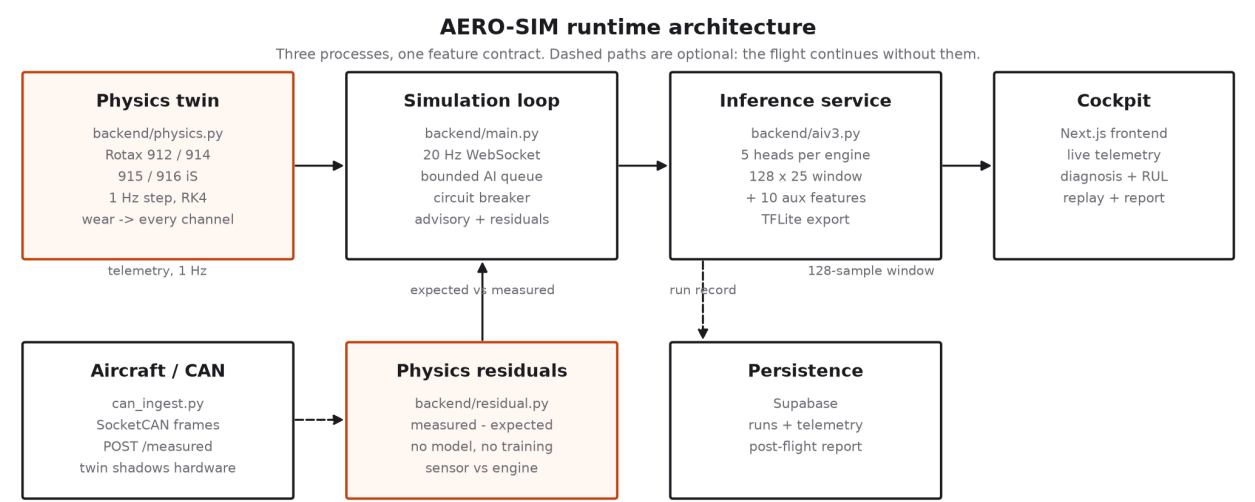


Figure 1 — Runtime architecture. Solid paths carry a flight; dashed paths are optional and their loss degrades one feature rather than the flight.

1. The problem

PS 26054 asks for a digital twin of the piston engines used in medium-altitude long-endurance UAVs, able to run in real time, detect faults as they develop, identify which sensor or subsystem is at fault, and estimate how much useful life the engine has left. The brief names four capability areas: engine and propulsion modelling; monitoring of the supporting systems including the electrical bus; detection of engine failure modes such as misfire, injector abnormality, cooling degradation and combustion instability; and sensor-fault handling, so that a lying sensor is not mistaken for a failing engine.

The system described here implements all four, with a live cockpit, an onboard-capable inference path, and a physics-based cross-check that shares nothing with the AI.

2. What the system is

Three processes cooperate through one shared feature contract (Figure 1).

- **The physics twin** (`backend/physics.py`) integrates the engine, propeller, airframe, thermal, oil, vibration and electrical behaviour of a chosen Rotax at 1 Hz, using RK4 inside each step. Accumulated wear feeds back into every observable channel, and four engine failure modes evolve as latent degradation processes.
- **The simulation loop** (`backend/main.py`) advances the twin, streams telemetry to the cockpit at 20 Hz, and forwards one sample per simulated second to the inference service through a bounded queue. Every stage is guarded: a stalled service, a bad physics step or a database outage degrades one feature, never the flight.
- **The inference service** (`backend/aiv3.py`) holds a rolling 128-sample window of 25 features per engine and runs five heads over it.
- **The cockpit** is a Next.js dashboard: live instruments, the AI's diagnosis, the advisory, the physics residuals, mission replay and a post-flight report.

A fourth element runs beside the AI rather than inside it. `residual.py` compares every sensor reading with what the physics expects at the current power, airspeed, altitude and wear. It has no weights and no training set, so when it agrees with the AI that agreement is independent evidence — and when it disagrees, the cockpit can say which of the two is out of step, and whether the engine or the sender is the likelier culprit.

3. The physics twin

The twin is not a lookup table of operating points. It integrates shaft dynamics against the propeller load, derives thermal states through first-order lags with their own time constants, and models oil pressure as a pump-and-relief-valve characteristic that falls with hot oil. Each engine carries its own torque curve, gear ratio, propeller and time-between-overhaul (TBO) figure.

3.1 Wear is visible in the sensors

The property that makes remaining-life estimation possible at all is that wear changes what the instruments read. Over a full life the twin applies: oil pressure down 30 percent, cylinder-head temperature up 25 °C, exhaust-gas temperature up 45 °C, oil temperature up 10 °C, vibration up 150 percent, brake-specific fuel consumption up 18 percent, and peak torque down 12 percent. Before this, a 1,900-hour engine read identically to a new one and the life estimate could only track elapsed time.

3.2 Failure modes and sensor faults are different things

Misfire, injector fouling, cooling degradation and combustion instability are modelled as engine behaviours with their own onset and growth: misfire drops firing cycles and shows as torque ripple and vibration; a fouled injector burns more fuel for the same power and shifts injection phasing; degraded cooling leaves heat in the engine in proportion to the heat being made; combustion instability raises cycle-to-cycle variance while leaving the mean alone. Separately, the sensors themselves can fail — bias, drift, spike, stuck-at, noise — armed by operating stress with hysteresis. Keeping the two apart is what lets the system answer “is the engine sick, or is the wire?”

4. The AI

Five heads per engine read the same 128 × 25 window, scaled by that engine's own fitted scaler. A TCN encoder feeds the four classification heads; remaining life has its own branch and, as Section 5 explains, its own dataset.

Head	Output	Result on held-out data
Detection	fault present (binary)	accuracy 95.3–96.6%, AUC 0.981–0.992
Diagnosis	8 channels × 6 fault types	per-channel fault type and confidence
Severity	0–100% per channel	regressed against injected stress
Failure modes	4 modes, independent	severity per mode, tuned present-thresholds
Remaining life	engine hours vs TBO	0.56–0.63% of TBO (see Section 6)

The window also carries ten auxiliary features, four of which are physics residuals computed from closed-form healthy baselines — oil-pressure ratio, vibration ratio, BSFC ratio and CHT excess. These are reproduced identically in the training pipeline and in the live service, and a parity test drives one row at a time through both to prove it.

5. Case study: why remaining-life estimation was failing

This section is included because it is the most instructive part of the work: the fault was not in the model, and no amount of training would have found it.

The deployed life head scored 22.9 hours of mean absolute error on its own test set, and in the cockpit it read 3.5–6.2 percent low on a fresh engine — consistently pessimistic, exactly where a demonstration starts. Retraining for 80 epochs changed nothing. Two measurements on the training data explained why.

- **The label moved inside the window.** The twin ages an engine fast enough to watch, so true remaining life fell by a median of 15.8 hours — p95 34.3 hours — across a single 128-sample window. The head's error was the same size as the ambiguity of its own target.
- **Fresh engines were nearly absent.** Only 3.7 percent of training rows sat in the top decile of engine life, because wear climbs steeply within every scenario. Every real flight begins there.

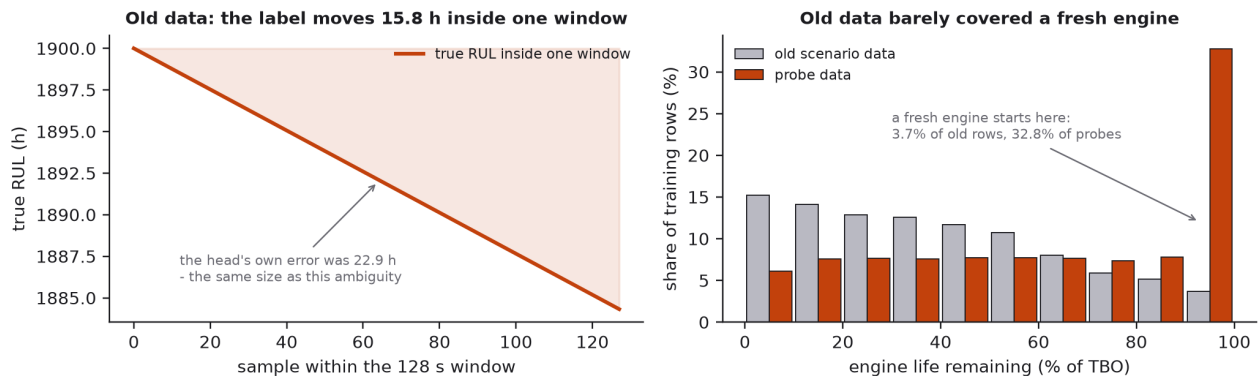


Figure 2 — Both panels are measured on the 914 data. Left: the target's own movement within one window. Right: life-stage coverage, old scenario rows against the probe dataset.

5.1 The fix was a dataset, not an architecture

A second dataset was built specifically for this head. Each sample is a *probe*: one engine held at one frozen life stage while it flies a short segment. Because every sensor channel is computed from wear before wear is updated, the recorded window and the label describe exactly the same engine, and the target is exact rather than smeared. Life stages are drawn uniformly with a deliberate over-sample of nearly-new engines, and the session clock is drawn independently of wear so the head cannot read the clock instead of the sensors — its correlation with the label is 0.000 by construction.

One further correction was necessary. The twin computes fuel flow directly from wear with no sensor error, which made one auxiliary feature an exact, invertible read of the answer (correlation -1.000). A head trained on that learns nothing else. Fuel-flow measurement error was therefore modelled — a per-installation calibration offset and per-sample transducer noise — leaving the feature informative but not decisive at -0.90 .

5.2 What the model contributes

On the new dataset a ridge regression over four summary statistics per channel reaches 0.69 percent of TBO. That is the floor any network must beat, so the deployed head is *initialised at it*: its linear path carries the ridge coefficients and the network's output starts at zero, reproducing the ridge to within 0.01 hours before training begins. Training can then only improve on it, and if it cannot, the notebook keeps the linear solution and says so.

This also answers a question worth asking directly — whether the recurrent branch earns its place. Trained under identical budgets on identical data:

Branch	Test MAE (914)	Verdict
LSTM alone	231.4 h (11.57% of TBO)	not competitive
Window statistics alone	16.5 h (0.83%)	already beats the previous head
Both together (deployed)	11.4 h (0.57%)	the LSTM earns its place only alongside

6. Results

6.1 Held-out test data

Engine	TBO	MAE	% of TBO	Near-new bias	Gates
912 ULS	2000 h	11.6 h	0.58%	−0.04%	all pass
914 UL/F	2000 h	11.4 h	0.57%	−0.07%	all pass
915 iS	1200 h	7.1 h	0.59%	+0.01%	all pass
916 iS	2000 h	12.5 h	0.63%	−0.01%	all pass

The acceptance gates are: error within 2 percent of TBO; near-new bias within ± 2 percent; correlation with true life at least 0.95; no inversion between life deciles; a beat over the ridge baseline; and continued usefulness with the fuel-flow feature zeroed.

6.2 Live flights

Offline scores can flatter a model. Each head was therefore flown in the running stack on an identical profile and scored against the twin's own true remaining life — the quantity the label is built from, with no proxy in between.

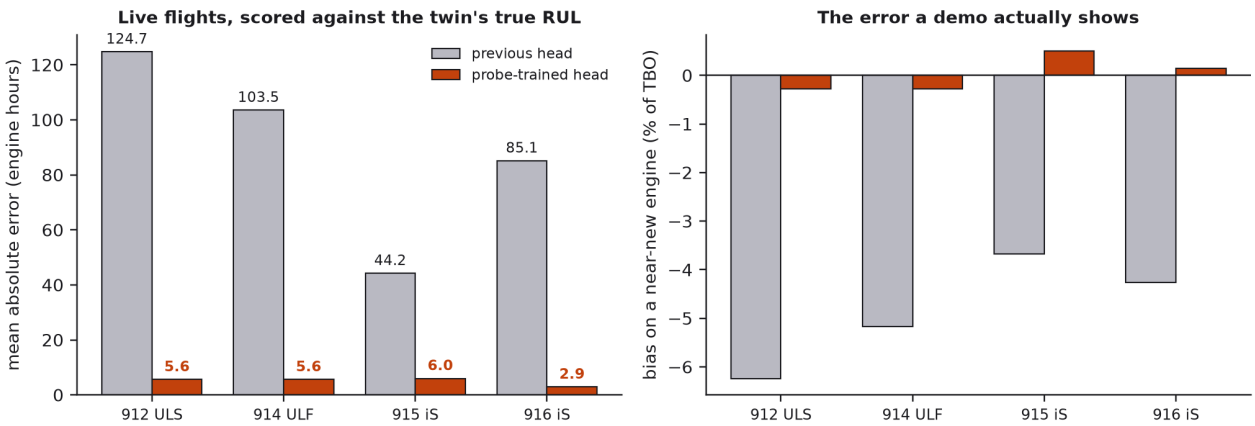


Figure 3 — Live flights, 100 samples per engine, previous head against the deployed one. Right: the bias a fresh-engine demonstration actually shows.

Engine	Previous head	Deployed head	Improvement
912 ULS	124.7 h (−6.24% of TBO)	5.6 h (−0.28%)	22.5×
914 UL/F	103.5 h (−5.17%)	5.6 h (−0.28%)	18.4×
915 iS	44.2 h (−3.68%)	6.0 h (+0.50%)	7.4×
916 iS	85.1 h (−4.26%)	2.9 h (+0.15%)	29.1×

The previous heads were biased low by 3.7 to 6.2 percent of TBO on every engine, which matches the symptom observed in the cockpit before any of this analysis began. The deployed heads sit within half a

percent, and two of them now err slightly high rather than always short.

7. Verification

7.1 The Simulink model is the same engine

The plant also exists as a Simulink model. Rather than assert equivalence, the two were compared step for step with every random draw frozen to its no-event value, so any difference would be a porting error rather than noise.

```
4 engines · 47 channels · 600 steps  
worst absolute error 4.97e-06, worst relative error 4e-10 – the reference file's own write  
precision
```

Reaching that required one modelling decision worth stating plainly: the Python twin advances in 1-second steps with RK4 inside each step, so the Simulink model is fixed-step discrete at 1.0 s and performs the same RK4 internally. A continuous solver at 0.01 s is a different integrator on the same equations and could be called equivalent, but not identical.

7.2 Automated tests and guards

- 36 backend tests cover the simulation loop's safety net — deliberate solver blow-ups, non-finite telemetry, AI outages — and the cockpit's display contract.
- Every deployed head is exported with a parity check against its training checkpoint (extract and reload both 0.0) and its SHA-256 recorded in a manifest.
- A live parity test drives real test scenarios one row at a time through the serving path and the training pipeline, comparing the window, the auxiliary features and all five outputs.

7.3 Defects found by flying it

Several faults were invisible to offline metrics and appeared only in a live cockpit. They are listed because they are the strongest argument for testing a twin by flying it:

- A zero value for the engine-failure-shutdown hold, documented as “disabled”, instead ended every flight at $t = 1$ s, healthy or not.
- A healthy engine was pinned at 70 percent health: the diagnosis head called the RPM sensor “stuck” at steady cruise, where a constant reading is exactly what a healthy sensor produces. The physics check already guarded against this; the health rule did not.
- Remaining life could read above TBO, and could *rise* during a flight while the window still held ground-roll samples. The cockpit now shows no figure at all until the window is clean, with a progress countdown, and the figure never rises thereafter.
- A crashed airframe reported a healthy engine, because the model's window describes the seconds before impact.

8. Limitations and honest gaps

- **No real engine data.** Every number here is measured against the physics twin. The twin is calibrated to published Rotax figures, but a twin verified against itself is not a twin verified against hardware. Residual tolerances in particular would need bench data.
- **Vibration fault injection is coarse.** One injection path rails the vibration channels against their sensor limits on roughly half the affected samples; correcting it needs a dataset regeneration, which has not been done.
- **Failure-mode onsets are rare by design** (about $1e-5$ per second), so a short run usually shows none. Judging those heads needs long runs or seeded wear.
- **The CAN path is untested on hardware.** The bridge and the endpoints work against a simulated aircraft; no adapter has been connected.
- **One cockpit behaviour is unflown.** Forcing health to zero on a crash is implemented and type-checked but has not been confirmed on screen.

9. Running the system

```
# all three services, in order, each waited for
scripts/start_stack.sh

# train a remaining-life head (one engine at a time: 8 GB machine)
scripts/train_rul_v3.sh 914

# ship it – swaps only the RUL head, leaves the other four untouched
validation/venv/bin/python3 validation/split_deployable_heads.py \
    --engines 914 --rul-phase phase5_rul_v3

# compare two heads on live flights against the twin's true RUL
scripts/rul_live_ab.py --engine Rotax_914_ULF --model new=... --model old=...
```

10. In one paragraph

AERO-SIM simulates four Rotax engines in enough physical detail that wear and failure modes change what the instruments read; five neural heads read that telemetry through one fixed contract and report faults, their location, their severity, the engine failure mode and the remaining life in engine hours; a model-free physics check runs alongside and can contradict them; and the whole path has been verified against held-out data, against live flights, and against an independent Simulink implementation of the same plant. The most valuable result was not a better model but a measurement: the remaining-life head had been asked to predict a target that moved by more than its own error inside a single observation window, and fixing the data — not the architecture — improved live accuracy by 7 to 29 times.